- At this point, we have a chain of Rack apps, each of which knows the identity of the next item in the chain. We trigger the chain by calling the call method on the topmost element in the chain. As part of being a good Rack citizen, each middleware app is expected to invoke the call method on its app argument. In this way, every Rack app is invoked.

Let's trace this through our sample Rack app. We start with run RackApplication.new, giving us an instance of RackApplication. The code then goes up a line, to use PostfixingMiddleware, and it creates an instance of PostfixingMiddleware, passing it the already created RackApplication instance. We repeat the process going upward, creating a PrefixingMiddleware instance that has a relationship with the PostfixingMiddlware instance.

Now that we have an entire chain of objects that conform to the Rack standard and respond to call, we invoke call on the object at the end of the chain, so our PrefixingMiddleware instance gets called with the env environment directly from the web request. The first thing that happens in that call method is status, headers, body = @app.call(env)—meaning we immediately call the next app in the chain. The first thing that happens in the next call method is the same status, headers, body = @app.call(env), which takes us down to the bottom Rack app, which responds to call and returns the three-element array [200, {"content-type" => "text/html"},["<h1>Welcome to Rack</h1>"]].

At this point, we walk back up the chain. That return value is extracted by the call method in PrefixingMiddleware which promptly takes the three values, prefixes some text to the body and returns a new three-element array with the new body. That return value is, in turn, extracted by PostfixingMiddleware, which does takes the values, postfixes some text to the body, and returns the new three-element array.

And, if you run rackup on this file and then hit http://localhost:9292 you will see all three lines of text in your browser, correctly formatted as HTML.

Although our Rack middleware classes both do the similar things—retrieve the previous call and adjust the body text, Rack middleware can do far more. You have complete freedom to adjust the environment, status, headers, and response body. You can completely ignore the work of other middleware and change the response to ["Never gonna give you up"]. More usefully, you can add API tokens to the environment, or perform external services like logging, or filter the response body in some way. There's a lot of power there.

And although we've been using rackup, Rack applications work with a variety of compatible Ruby web servers. To use the popular Puma server, just:

```
$ gem install puma
$ puma
```

Puma will, by default, look for a config.ru file in the current directory and run it.

Every major Ruby web framework is structured as a Rack application that can be triggered with Rack's run. Not only can your middleware be run with any web server, it can be integrated into any web framework. Any problem that can be solved with a Rack middleware can be applied to any compatible framework.

Sinatra and Web Frameworks

While Rack is very powerful, it's also a little low-level and writing more complex web interactions with it is complicated. For more involved web applications, Ruby provides many